

UNIDAD 5:

PYTHON

AVANZADO

CURSO

IA

```
01 import openai
02 from langchain import
03     ChatOpenAI
04
05 def generar_respuesta(prompt):
06     modelo = ChatOpenAI(
07         model = "gpt-4o"
08         temperature = 0.7
09     )
10
11     respuesta = modelo.invoke(
12         prompt
13     )
14     return respuesta.content
15
16
17 if __name__ == "__main__":
18     prompt = "Explicame qué
19     es la inteligencia artificial"
20     print(generar_respuesta(prompt))
```



Docente:
Gabriel Mosso



BRAINSTORM

Rep.Argentina 786, Sunchales, Sta fe.



GabrielNMosso@GMail.com



Tel: +54 351 3733383



Python Bucles y Estructuras condicionales:

Los bucles y las estructuras condicionales son dos tipos de estructuras de control en la programación que permiten controlar el flujo de un programa, pero tienen propósitos y comportamientos diferentes.

Estructuras Condicionales:

Propósito: Las estructuras condicionales se utilizan para tomar decisiones en el código, ejecutando ciertas partes del mismo solo si se cumplen ciertas condiciones.

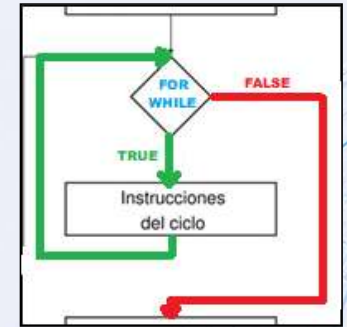
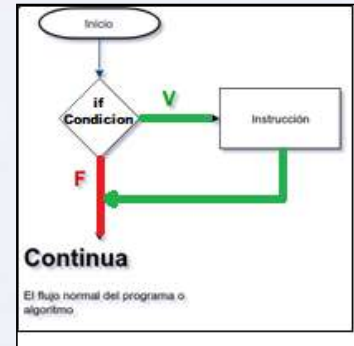
Funcionamiento: Evalúan una o más expresiones booleanas (verdadero o falso) y ejecutan bloques de código según el resultado.

Uso: Se utilizan cuando necesitas ejecutar código de forma selectiva en función de ciertas condiciones.

Bucles (Loops):

Propósito: Los bucles se utilizan para repetir la ejecución de un bloque de código varias veces, basándose en una condición o en una secuencia de elementos.

Funcionamiento: Repiten un bloque de código mientras se cumpla una condición (while) o iteran a través de una secuencia (for).



```
#Estructura if
edad = 20
if edad >= 18:
    print("Eres mayor de edad")
else:
    print("Eres menor de edad")
```



```
# Bucle for
for i in range(5):
    print("Este es el número", i)
```

```
#Bucle While
contador = 0
while contador < 5:
    print("Contador:", contador)
    contador += 1
```



Estructura condicional (IF):

La estructura if en Python es una de las formas más fundamentales de control de flujo. Se utiliza para ejecutar un bloque de código solo si una condición específica es verdadera.

Componentes de la Estructura if

if: Palabra clave que indica el inicio de una estructura condicional. Debe ir seguida de una condición (una expresión que se evalúa como True o False).

Condición: Es una expresión que Python evalúa. Si el resultado es True, se ejecuta el bloque de código debajo del if. Si es False, el bloque se omite. La condición puede ser cualquier expresión que devuelva un valor booleano (True o False), como una comparación ($x > 5$).

Bloque de Código: Es el conjunto de instrucciones que se ejecutan si la condición es verdadera. (debe estar indentado)

elif (else if): Se utiliza para evaluar una nueva condición si la condición del if inicial es falsa. Puedes tener múltiples elif en una estructura condicional. Solo se ejecuta el bloque asociado al primer elif que sea verdadero.

else: Es opcional y se coloca al final de la estructura condicional. Se ejecuta si ninguna de las condiciones anteriores (en if o elif) es verdadera. No requiere condición, ya que actúa como un caso de "en cualquier otro caso".

Indentación: La indentación es crucial en Python. Cada bloque de código dentro de un if, elif o else debe estar indentado al mismo nivel. Un error común es olvidar la indentación o tener niveles inconsistentes, lo que causará errores de sintaxis.

```
if condición_1:
    # Bloque de código a ejecutar si condición_1
    es verdadera
elif condición_2:
    # Bloque de código a ejecutar si condición_1 es falsa
    y condición_2 es verdadera
else:
    # Bloque de código a ejecutar si todas las
    condiciones anteriores son falsas

#-----

hora = 10
dia = "Lunes"

if hora < 12 and dia == "Lunes":
    print("Es lunes por la mañana.")
elif hora >= 12 or dia == "Domingo":
    print("Es la tarde o es domingo.")
else:
    print("Es otro momento de la semana.")
```





BUCLE (FOR):

La estructura for en Python es una herramienta fundamental para iterar sobre secuencias como listas, tuplas, cadenas de texto, diccionarios, conjuntos y más. A diferencia de otros lenguajes que usan un contador para iterar, en Python el for se basa en iterar directamente sobre los elementos de una secuencia.

Componentes de la Estructura for

for: Es la palabra clave que inicia el bucle. Indica el comienzo de la iteración sobre una secuencia.

elemento: Es una variable temporal que toma el valor de cada elemento en la secuencia, uno por uno, en cada iteración del bucle.

in: Es una palabra clave que conecta la variable del elemento con la secuencia que se va a iterar. Indica que la iteración se hará sobre los elementos de la secuencia.

secuencia: Es el conjunto de elementos sobre los que se itera. Puede ser una lista, tupla, cadena de texto, conjunto, diccionario, o cualquier objeto iterable.

Bloque de Código: Es el conjunto de instrucciones que se ejecuta en cada iteración del bucle.

Interrupción de un Bucle con break y continue:

break: Termina el bucle antes de que todas las iteraciones se completen.

continue: Salta a la siguiente iteración del bucle sin ejecutar las líneas de código restantes en la iteración actual.



```
for elemento in secuencia:
    # Bloque de código a ejecutar para cada paso en la secuencia
#-----
frutas = ["manzana", "banana", "cereza"]
for fruta in frutas:
    print(fruta)
#-----
numeros = [1, 2, 3, 4, 5]
for numero in numeros:
    print(numero * 2)
#-----
persona = {"nombre": "Ana", "edad": 30, "ciudad": "Madrid"}
for clave, valor in persona.items():
    print(f"{clave}: {valor}")
#-----
for i in range(10):
    if i == 5: # Imprimirá del 0 al 4 y luego se detendrá
        break
    print(i)
#-----
for i in range(5): # Imprimirá 0, 1, 2, 4 (se salta el 3)
    if i == 3:
        continue
    print(i)
```



BUCLE (WHILE):

El bucle while en Python es una estructura de control que permite ejecutar un bloque de código repetidamente mientras una condición sea verdadera. A diferencia del bucle for, que itera un número determinado de veces.

Componentes del Bucle while

while: Es la palabra clave que inicia el bucle. Indica que el bloque de código siguiente se repetirá mientras la condición se mantenga verdadera.

Condición: Es una expresión que se evalúa antes de cada iteración del bucle. Mientras esta condición sea True, el bloque de código dentro del bucle se ejecutará.

Bloque de Código: Es el conjunto de instrucciones que se ejecuta en cada iteración del bucle. (debe estar indentado)

Interrupción del Bucle con break y continue

break: Termina inmediatamente el bucle, independientemente de si la condición sigue siendo verdadera.

continue: Salta el resto del bloque de código en la iteración actual y vuelve a evaluar la condición al inicio del bucle. Se utiliza cuando quieres omitir ciertas iteraciones pero continuar con el bucle.

Precauciones con el Bucle while

Bucles Infinitos: Si la condición del while nunca se vuelve falsa, el bucle continuará indefinidamente, lo que puede causar que el programa se cuelgue. Es importante asegurarse de que algo en el bloque de código del bucle modifique la condición, o de lo contrario, podrías terminar en un bucle infinito.

```
while condición:
    # Bloque de código a ejecutar mientras la condición
    # sea verdadera
    #-----
    contador = 0

while contador < 5:
    print("El contador es:", contador)
    contador += 1
    #-----
    contador = 0

while contador < 10:
    print("Contador:", contador)
    if contador == 5: # Imprimirá los valores de 0 a 5 y luego
        # saldrá del bucle
        break
    contador += 1
    #-----
    contador = 0

while contador < 5:
    contador += 1
    if contador == 3: # Imprimirá 1, 2, 4, 5 (se salta el 3)
        continue
    print("Contador:", contador)
```



TRY (MANEJO DE EXCEPCIONES):

La estructura try en Python es una parte fundamental del manejo de excepciones, que permite controlar y gestionar errores en el código de manera elegante. En lugar de permitir que el programa falle cuando ocurre un error, try te permite capturar ese error y ejecutar un código alternativo o tomar alguna medida para manejar la situación.

Componentes de la Estructura try

try: Inicia un bloque de código donde se espera que pueda ocurrir una excepción. Si el código dentro del bloque try genera una excepción, la ejecución se transfiere al bloque except correspondiente.

except: Define el bloque de código que se ejecuta si ocurre una excepción en el bloque try. Puedes especificar el tipo de excepción que deseas manejar (por ejemplo, ValueError, ZeroDivisionError, etc.), o puedes manejar todas las excepciones sin especificar un tipo.

Es posible tener múltiples bloques except para manejar diferentes tipos de excepciones de manera distinta.

Manejo de Múltiples Excepciones

Puedes manejar diferentes tipos de excepciones utilizando varios bloques except, uno para cada tipo de excepción que deseas manejar.

```
try:
    numero = int(input("Introduce un número: "))
    resultado = 10 / numero
except ValueError:
    print("Error: Debes introducir un número.")
except ZeroDivisionError:
    print("Error: No se puede dividir entre cero.")
except Exception as e:
    print("Error inesperado: " + e)
```





EJERCICIO:

Vamos a escribir un programa que solicite al usuario ingresar una serie de números.

El programa debe:

1-Continuar solicitando números hasta que el usuario decida detenerse ingresando la palabra "salir", los números deben ir guardandose en una LISTA.

(Si el usuario introduce un valor no numérico, debe manejarse la excepción.)

2-Al finalizar la carga el programa debe:

2.1-Utilizar un bucle FOR para mostrar la suma de los números introducidos.

2.2- Calcular el promedio de la LISTA, utilizando la función "len" que devolverá la cantidad de números que tiene la lista.

(Si el usuario introduce al menos un número.)

```
numeros = [] #Lista vacía

while True:
    try:
        entrada = input("Introduce un número (o 'salir' para terminar): ")

        if entrada.lower() == 'salir':
            break

        numero = float(entrada)
        numeros.append(numero)

    except ValueError:
        print("Eso no es un número válido. Inténtalo de nuevo.")

print("Ingreso completado...")

if len(numeros):
    # Calcular la suma manualmente con un bucle for
    suma = 0
    for numero in numeros:
        suma = suma + numero #suma += numero
    promedio = suma / len(numeros)

    print("La suma de los números es: " + str(suma))
    print("El promedio de los números es: " + str(promedio))
else:
    print("No se ingresaron números válidos.")
```



Familiarización con Python via Jupyter Notebook.

Ejecución WEB de Jupyter Notebook:

- Ingresar a la web >> <https://jupyter.org/try-jupyter/lab/>
- Arrastrar el archivo >> "Tutorial de Python con Jupyter Notebook.ipynb" a la izquierda de la web.

NOTA: Esto provocará una ejecución lenta del código, ya que se estará ejecutando en un servidor de Jupyter.

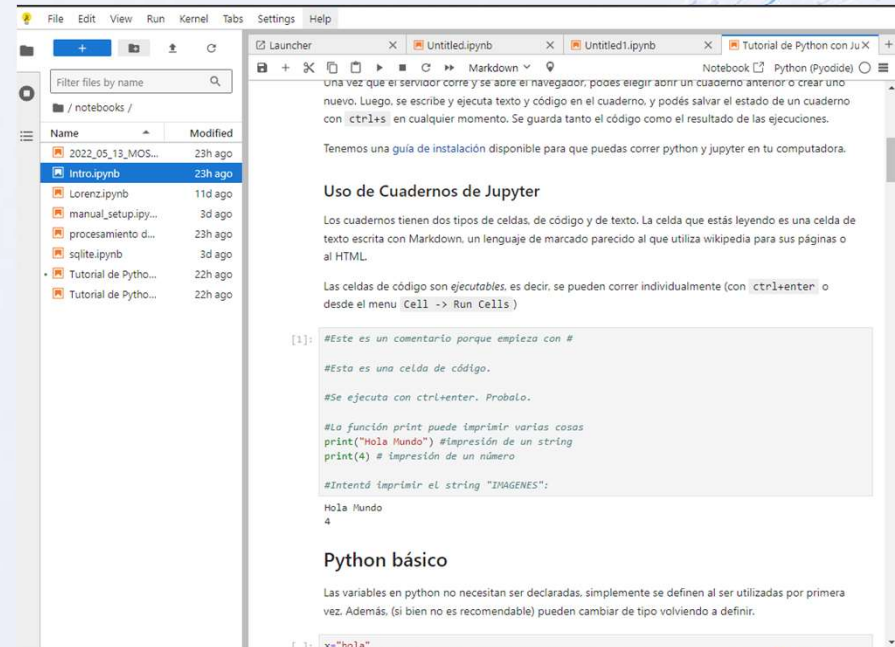
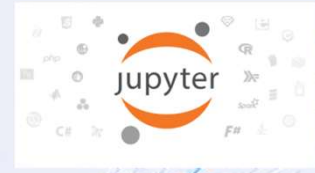
Ejecución LOCAL de un archivo .IPYNB:

- Copiar el archivo "Tutorial de Python con Jupyter Notebook.ipynb" en "C:/"
- Abrir el CMD y ejecutar la siguiente línea de código:
- **jupyter notebook "c:\Tutorial de Python con Jupyter Notebook.ipynb"**

NOTA: Esto provocará una ejecución rápida del código, ya que se estará ejecutando localmente utilizando los recursos de la PC.

>> Ejecutar las celdas de código y realizar pruebas para obtener diferentes resultados.

>> Copiar el código a VSCODE y ejecutarlo desde allí para familiarizarse con ambos entornos IDE.





```
> print("Gracias !")
> print("dudas, consultas...")
> input("Presione una tecla para salir...")
```



GabrielNMosso@GMail.com



Tel: +54 351 3733383



Docente:
Gabriel Mosso



BRAINSTORM

Rep. Argentina 786, Sunchoales, Sta. Fe.